Temporal Botnet Detection using Graph Neural Networks (GNNs) & SMOTE Augmentation

Table of Contents

Temporal Botnet Detection using Graph Neural Networks (GNNs) & SMOTE Augmentation

1. Fardin Rahman 2. Jubayer Alam Likhon

1. Abstract

Internet of Things (IoT) network security faces severe challenges from botnet intrusions—such as Distributed Denial of Service (DDoS), Denial of Service (DoS), Reconnaissance, and Data Theft. Attacks such as DDoS exhibit complex host interaction topologies, evolving temporal dynamics. Traditional machine learning intrusion detection systems evaluate individual flow records as independent tabular instances, failing to capture topological communication structures and transient behavioral transitions. In this paper, we propose an end-to-end spatial-temporal Graph Neural Network (GNN) framework for multi-class IoT botnet intrusion detection evaluated on the complete 5% UNSW 2018 Bot-IoT dataset comprising 3,668,522 network flows. Network traffic is modeled as a sequence of dynamic snapshot graphs $G_t = (V, E_t, X_t, E_{\text{attr},t})$ sliced across 20 temporal windows, mapping IP host addresses to graph nodes with dynamic degree metrics and associating 23 standardized continuous flow attributes to directed edges. Synthetic Minority Over-sampling Technique (SMOTE) is applied strictly to training edges to mitigate class imbalance without evaluation leakage. We systematically compare Track A (a tabular Flow Multi-Layer Perceptron baseline) against Track B (Spatial GCN, Spatial GraphSAGE, and a hybrid Temporal LSTM-GNN). The Temporal LSTM-GNN achieves state-of-the-art performance with 98.64% accuracy, a 99.04% weighted F1-score, and a 0.9995 macro ROC-AUC, significantly outperforming the tabular baseline (56.79% accuracy). Inferential hypothesis testing demonstrates a statistically significant error reduction across temporal snapshots ($t = 11.70, p = 3.97 \times 10^{-10}$), establishing the necessity of joint spatial-temporal graph abstractions for IoT security.

2. Introduction

2.1 Problem Statement

The rapid expansion of Internet of Things (IoT) ecosystems—spanning smart home appliances, industrial sensors, and autonomous infrastructure—has dramatically enlarged the modern cyber-attack surface. IoT devices are particularly vulnerable to Cyber Attacks, back door to a network and recruitment into large-scale botnets (e.g., Mirai, Bashlite), due to hardware constraints and lightweight firmware,. Contemporary IoT botnet operations execute multi-stage attack lifecycles, beginning with passive and active network probing (Reconnaissance: OS fingerprinting and service scanning), progressing to lateral movement and credential harvesting (Keylogging and Data Theft), and culminating in high-volume flooding attacks (Distributed Denial of Service [DDoS] and Denial of Service [DoS]).

Detecting these intrusive behaviors in real time is hindered by two primary operational challenges:

1. **Severe Class Imbalance**: In operational network environments, high-volume volumetric attacks (DDoS and DoS) comprise over 97% of total packet flows, whereas critical early-stage intrusive indicators such as Reconnaissance (~2.5%), Normal operational traffic (~0.01%), and data exfiltration (Theft, ~0.002%) occur in negligible proportions. Standard machine learning loss functions naturally overfit to the majority classes, collapsing minority class recall.

2. **Tabular Isolation vs. Topological Reality**: Traditional Network Intrusion Detection Systems (NIDS) rely on tabular machine learning classifiers (e.g., Decision Trees, Random Forests, Multi-Layer Perceptrons) that treat each netflow tuple as an independent, identically distributed vector. This isolated tabular view completely ignores the structural communication topology (e.g., fan-in degree bursts targeting victim hosts during DDoS vs. fan-out port scanning during Reconnaissance) and temporal persistence across consecutive traffic windows.

2.2 Real-World Motivation

Real-world IoT botnet campaigns depend intrinsically on graph communication patterns. A single host performing port scans displays low individual flow byte counts but exhibits high out-degree variance across multiple destination IP addresses over short time intervals. On the other side, a victim node under a DDoS attack receives overwhelming in-degree traffic volume from thousands of compromised bot agents. Evaluating individual flows without topological context prevents classifiers from distinguishing malicious low-rate probing from benign transient bursts. Furthermore, tracking temporal state transitions across dynamic graph snapshots allows defense systems to identify early-stage reconnaissance patterns and isolate compromised nodes before destructive high-volume DDoS attacks are unleashed.

2.3 Research Questions

This investigation addresses three central research questions:

RQ1: *Topological Representation Advantage* — Does formulating network flow data into graph structures significantly enhance multi-class botnet intrusion detection performance over isolated tabular feature classification?

RQ2: *Temporal Memory Integration* — How does combining inductive spatial graph convolutions with recurrent temporal state tracking (LSTM) affect classification accuracy and minority class detection across dynamic network snapshots?

2.4 Explicit Objectives and Key Contributions

To answer these questions, this paper makes the following key contributions:

1. **Dynamic Graph Formulation of Full IoT Flow Traffic**: We model the complete 5% UNSW 2018 Bot-IoT dataset (3,668,522 flows) as a dynamic sequence of $T = 20$ temporal snapshot graphs $G_t = (V, E_t, X_t, E_{\text{attr},t})$, mapping unique IP host addresses to graph nodes equipped with dynamic topological features ($d_{\text{in}}, d_{\text{out}}, P_{\text{src}}, P_{\text{dst}}$) and attaching 23 standardized flow attributes to directed edges.

2. **Leakage-Free Graph-Based SMOTE Oversampling**: We establish a strict edge-mask partitioning protocol where SMOTE oversampling is executed exclusively on training edge features ($is\_train == True$), raising minority class counts (Reconnaissance to 100k, Normal to 50k, Theft to 50k) while maintaining pristine evaluation test splits (733,706 flows).

3. **Dual-Track Architectural Benchmark**: We design, implement, and benchmark four distinct neural network architectures categorized into Track A (Tabular Flow MLP Baseline) and Track B (Spatial GCN, Inductive Spatial GraphSAGE, and Spatial-Temporal LSTM-GNN).

4. **Rigorous Inferential Statistical Validation**: We evaluate model performance using comprehensive multi-class metrics (Macro/Weighted F1, ROC-AUC) and execute formal paired sample t-tests and Wilcoxon signed-rank tests across temporal snapshots, demonstrating statistically significant performance superiority ($t = 11.70, p = 3.97 \times 10^{-10}$).

3. Related Work and Background

3.1 Statistical Methods in Network Anomaly Detection

Statistical characterization of continuous network traffic flows forms the foundation of non-parametric anomaly detection. Linear feature correlation analysis via Pearson's coefficient (r_{xy}) identifies redundant dimensional attributes, such as collinearity between packet counts (pkts) and byte volumes (bytes) (Figure 1).

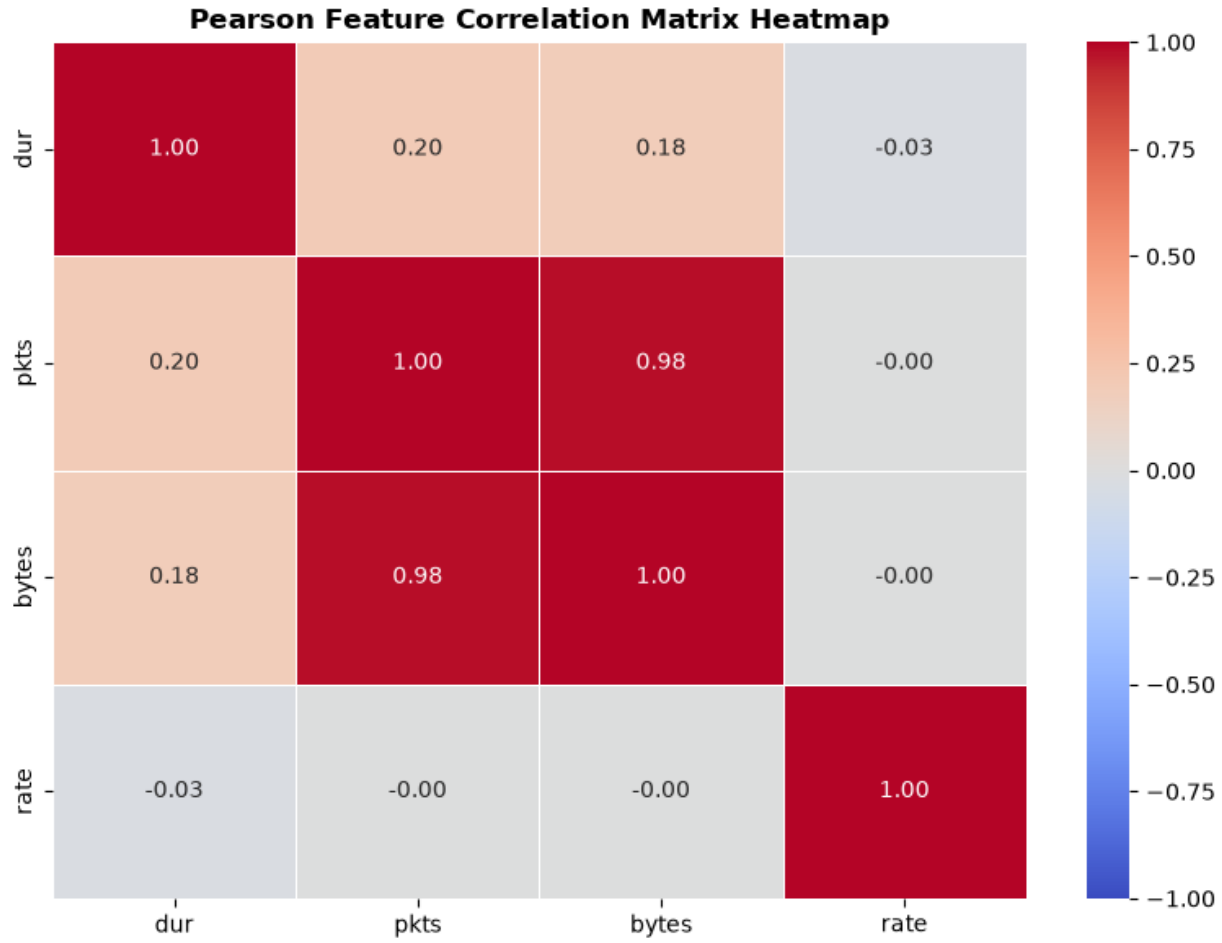

Figure 1: Pearson Feature Matrix Heatmap

However, network attacks frequently exhibit non-linear feature interactions that linear correlation fails to detect. **Mutual Information ($I(X;Y)$)** (Figure 2) quantifies the non-linear shared information between continuous flow attributes X and discrete attack categories Y see Table 1 for the score:

$$I(X;Y) = \sum_{y \in Y} \int_{X} p\,(x,y) \log \frac{p(x,y)}{p(x)p(y)} dx$$

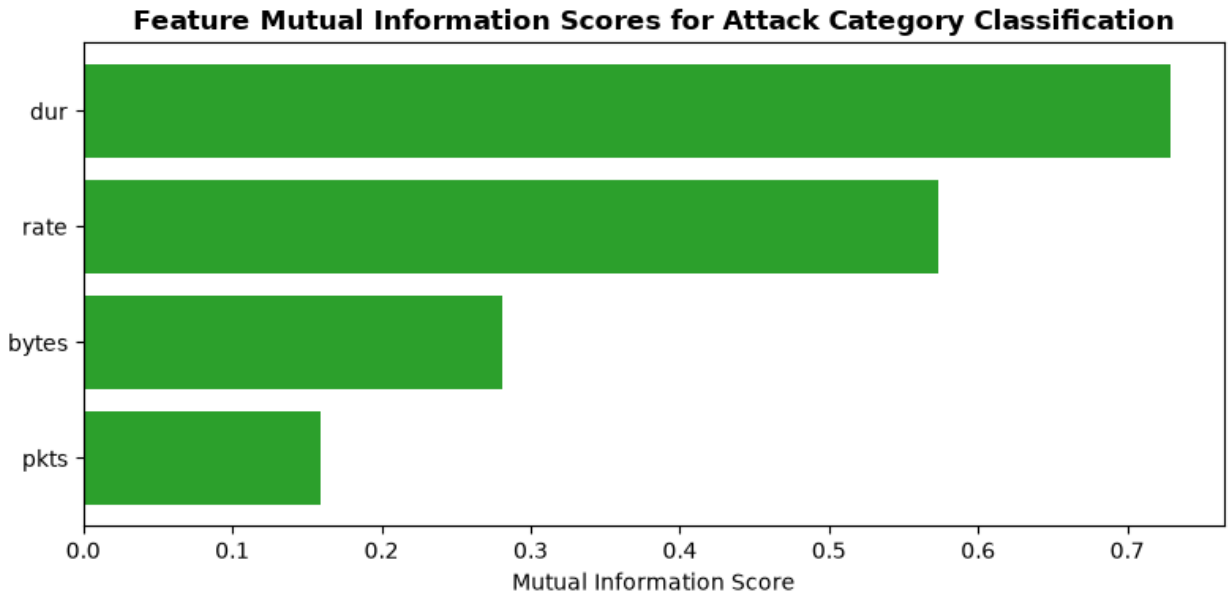

Figure 2: Mutual Information Score

Feature Variable	Mutual Information Score ($I(X;Y)$)	Predictive Power Rank
dur	**0.728517**	1 (Primary Indicator)
rate	**0.573337**	2 (Secondary Indicator)
bytes	**0.280645**	3
pkts	**0.159326**	4

Table 1: Mutual Information Score

In time-series analysis of packet rates, stationarity testing via the **Augmented Dickey-Fuller (ADF)** test evaluates the presence of unit roots in temporal flow aggregations:

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \sum_{i=1}^{p} \delta_i \, \Delta y_{t-i} + \epsilon_t$$

Rejecting the null hypothesis ($H_0: \gamma = 0$) establishes time-series stationarity, see Table 2, justifying temporal windowing and snapshot binning strategies for dynamic graph modeling.

Name	Value	
ADF Statistics	-79.906394	
P-value	0.00	
Lags Used	42	
Number of Observations	303634	
Critical Values	1%	-3.430372
	5%	-2.861550
	10%	-2.566775

Table 2: Augmented Dickey-Fuller (ADF) Stationarity Test

3.2 Graph Neural Network (GNN) Family

Graph Neural Networks extend deep learning operations to non-Euclidean domain structures $G = (V, E)$.

1. **Graph Convolutional Networks (GCN)**: Spectral Graph Convolutions [1] was introduced so that approximate first-order localized Chebyshev polynomials on graph Laplacians:

$$H^{(l+1)} = \sigma\left(\widetilde{D}^{-\frac{1}{2}}\tilde{A}\widetilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right)$$

   where $\tilde{A} = A + I_N$ represents the adjacency matrix with added self-loops, and $\widetilde{D}$ is the diagonal degree matrix. In edge classification tasks, node representations h_u, h_v generated by GCN layers are concatenated with raw edge feature attributes e_{uv} to predict link labels.

2. **Inductive GraphSAGE**: GraphSAGE [2] was introduced to perform inductive representation learning on evolving graphs using uniform neighborhood sampling and aggregation functions:

$$h_v^{(l+1)} = \sigma\left(W^{(l)} \cdot \left[h_v^{(l)} \parallel \text{AGGREGATE}_k\left(\{h_u^{(l)}, \forall u \in \mathcal{N}(v)\}\right)\right]\right)$$

   GraphSAGE enables zero-shot generalization to unseen nodes and dynamically changing graph topologies without requiring full-graph Laplacian recomputation.

3. **Spatial-Temporal Graph Neural Networks**: To process time-varying graphs $[G_1, \ldots, G_T]$, spatial GNN layers (GCN or GraphSAGE) extract localized structural node embeddings H_t^{spatial} within each snapshot t, while Recurrent Neural Network (RNN) memory cells—such as Long Short-Term Memory (LSTM) units—pass hidden state vectors $h_{v,t}$ across consecutive temporal snapshots:

$$(h_{v,t}, c_{v,t}) = \text{LSTMCell}\left(h_{v,t}^{\text{spatial}}, (h_{v,t-1}, c_{v,t-1})\right)$$

This hybrid formulation tracks joint spatial communication topology and dynamic temporal evolution.

3.3 Related Works

Table 3 summarizes relevant prior literature on the UNSW 2018 Bot-IoT dataset and related IoT intrusion detection benchmarks.

Reference	Dataset / Size	Feature Abstraction	Model Architecture	Key Limitations
[3]	UNSW 2018 Bot-IoT (Full)	10-best tabular features	SVM, RNN, Decision Trees	Evaluated tabular flows in isolation; collapsed on severe minority classes (Normal, Theft).
[4]	Bot-IoT / N-BaIoT	Static Graph Topology	E-GraphSAGE (Edge GNN)	Modeled network as a single static graph; failed to capture temporal snapshot transitions over time.
[5]	IoT intrusion-detection benchmark datasets; evaluates IoT network traffic	Feature-vector representation; local and surrounding feature information	Parallel Deep Autoencoder (PDAE)	Designed primarily for efficient feature learning from tabular traffic. It does not explicitly exploit communication topology or temporal graph evolution.
[6]	UNSW-NB15 + BoT-IoT	Selected network-flow features; sequential feature representation	Stacked LSTM, BiLSTM, LSTM Autoencoder and DNN variants	Models temporal/sequential dependencies but still treats traffic primarily as sequences/features rather than explicitly modeling host-to-host graph structure.
[7]	**BoT-IoT:** >72 million records; also ISOT	**Attribute graph:** traffic attributes are	Meta-path-based GNN / graph	Builds a graph around **attribute relationships** rather

Reference	Dataset / Size	Feature Abstraction	Model Architecture	Key Limitations
		represented as graph nodes	convolution for anomaly detection	than explicitly modeling the evolving communication topology between network entities.
[8]	IoT intrusion-detection traffic; designed for label-limited settings	Flow topology + traffic features	Topology-based GCN	Improves graph-based representation under limited labels, but the approach still does not fully address continuous **dynamic graph snapshot transitions** and severe rare-class imbalance. The paper is particularly relevant to your topology-based motivation.
[9]	Network-flow NIDS datasets	Network flows represented as graphs; learns graph representations without requiring all flows to be manually labeled	Self-supervised GNN	Reduces dependence on labels, but focuses on graph representation learning rather than explicitly modeling **temporal evolution of dynamic network graphs**.
[10]	Four NIDS benchmark datasets	Flow/IP information represented as graph nodes and edges; incorporates edge features	Edge-Directed Graph Multi-Head Attention Network	Improves edge-aware attention, but its focus remains on graph-level/topological relationships rather than modeling **time-varying graph snapshots**.
[11]	NF-BoT-IoT, NF-BoT-IoT-v2, NF-CSE-CIC-IDS2018,	Network flows represented using	GAT encoder + graph	Strongly reduces dependence on labeled data, but the

Reference	Dataset / Size	Feature Abstraction	Model Architecture	Key Limitations
	NF-CSE-CIC-IDS2018-v2	edge features and local graph topology	contrastive/self-supervised learning	formulation is still primarily graph/subgraph based and does not explicitly focus on **long-term dynamic snapshot transitions**.
[12]	Network-flow intrusion detection datasets	**Dynamic graph representation** of network traffic; host interactions and topology changes	Dynamic-graph GNN + integrated neural components	Explicitly addresses changing topology, making it close to your problem. However, dynamic graph modeling alone does not necessarily solve **extreme class imbalance / rare attack detection**, leaving room for imbalance-aware temporal modeling.
[13]	BoT-IoT, CICIDS, UNSW-NB15, ToN-IoT	KNN graph + LOF-based local anomaly features	GCN + Local Outlier Factor (LOF)	Specifically improves detection of sparse/rare anomalies, but the graph is constructed using KNN relationships rather than modeling **real temporal communication snapshots**.
Our Work	**UNSW 2018 Bot-IoT (Full: 3.67M)**	**Dynamic Graph** $G_t = (V, E_t, X_t, E_{\text{attr},t})$	**Spatial-Temporal SAGE-LSTM + Train SMOTE**	**Full 5-class multi-class evaluation; leakage-free edge SMOTE; formal inferential statistical validation.**

Table 3: Comparative survey of prior works on IoT botnet intrusion detection.

4. Dataset Description

4.1 Dataset Source and Capture Environment

The empirical evaluation in this study utilizes the **UNSW 2018 Bot-IoT dataset**, created by the Cyber Range Lab of UNSW Canberra (5% Dataset/All_features/UNSW_2018_IoT_Botnet_Full.csv). The dataset was generated within a realistic hybrid IoT network environment combining physical IoT devices (e.g., smart fridges, weather stations) and simulated nodes orchestrated via Node-RED. Network flow attributes were captured using the Argus network monitoring tool and converted into structured flow records spanning multi-day recording sessions.

4.2 Dataset Volume and Subset Rationale

Unlike many prior studies that restrict evaluation to the sub-sampled 5% subset - "10-best features" partition, this research ingests the **complete, 5% full feature UNSW 2018 Bot-IoT dataset**.

Total Flow Records: 3,668,522 netflow entries.

Original Dimensionality: 46 feature columns.

Storage Footprint: ~1.1 GB uncompressed CSV.

Rationale for Using the Full Dataset: Sub-sampled partitions artificially alter host interaction topologies and distort continuous inter-arrival time distributions. Utilizing the untruncated 3,668,522 flow records preserves authentic host communication graph structures, realistic traffic burstiness, and raw class imbalance ratios necessary to validate real-world operational NIDS performance.

4.3 Feature Inventory and Types

We extract 23 continuous numerical flow attributes and categorical variables to construct edge attributes (E_{attr}), as detailed in Table 4.

Feature Category	Variable Names	Data Type	Description
Volumetric Totals	pkts, bytes, spkts, dpkts, sbytes, dbytes	Integer / Continuous	Total, source, and destination packet counts and byte volumes per flow.
Temporal Duration	dur, mean, stddev, sum, min, max	Continuous (Seconds)	Flow duration and statistical distribution of sub-flow active/idle durations.

Feature Category	Variable Names	Data Type	Description
Traffic Rates	rate, srate, drate	Continuous (Pkts/Sec)	Overall flow transmission rate, source sending rate, and destination receiving rate.
Window Aggregations	TnBPSrcIP, TnBPDstIP, TnP_PSrcIP, TnP_PDstIP	Continuous	Total bytes/packets per source and destination IP over 100-flow sliding windows.
Connection Density	N_IN_Conn_P_DstIP, N_IN_Conn_P_SrcIP	Integer	Inbound connection counts per source and destination IP address.
Protocol / State	state_number, proto_number	Categorical Integer	Numerical encodings for network protocol (TCP, UDP, ICMP) and connection state (CON, FIN, INT).

Table 4: Continuous and encoded feature variables utilized for edge attribute vectors ($E_{attr} \in \mathbb{R}^{23}$).

4.4 Class Label Distributions and Missing Data Profile

The dataset defines a 5-class target variable (category) and an 8-class granular subcategory (subcategory), exhibiting extreme class imbalance as detailed in Table 5.

Major Category	Flow Record Count	Percentage (%)	Subcategories Included	Subcategory Counts
DDoS	1,926,624	52.52%	UDP, TCP, HTTP	UDP: 1,225,242 | TCP: 700,774 | HTTP: 608
DoS	1,650,260	44.98%	UDP, TCP, HTTP	UDP: 755,988 | TCP: 892,406 | HTTP: 1,866
Reconnaissance	91,082	2.48%	Service_Scan, OS_Fingerprint	Service_Scan: 73,168 | OS_Fingerprint: 17,914
Normal	477	0.013%	Normal	Normal: 477

Major Category	Flow Record Count	Percentage (%)	Subcategories Included	Subcategory Counts
Theft	79	0.002%	Keylogging, Data_Exfiltration	Keylogging: 73 | Data_Exfiltration: 6
Total	**3,668,522**	**100.00%**	**8 Subcategories**	**3,668,522 Total Flows**

Table 5: Raw class and subcategory distribution across the full UNSW 2018 Bot-IoT dataset

.

Missing-Data Profile: Missing values (NaN) occurring within window aggregation features (TnBPSrcIP, TnBPDstIP) were imputed with 0. No rows were discarded due to missing values, retaining all 3,668,522 records.

5. Statistical Analysis

5.1 Descriptive Statistics and Higher-Order Distribution Moments

To establish an empirical baseline across continuous traffic variables, we compute central tendency, dispersion, higher-order statistical moments (Skewness γ_1 and Kurtosis γ_2), and Interquartile Range (IQR) outlier profiles, as presented in Table 6.

Feature Variable	Mean	Std Dev	Min	Median (50%)	Max	Skewness (γ_1)	Kurtosis (γ_2)	IQR Outliers Count (%)
dur (Flow Duration)	20.3348	21.4876	0.0000	15.5085	2,771.49	41.09	2,745.13	80,161 (2.19%)
pkts (Packet Count)	7.7260	115.5876	1.0000	7.0000	70,057.00	458.90	234,744.04	140,466 (3.83%)
bytes (Byte Volume)	869.0501	112,266.67	60.0000	600.0000	7.18×10^7	494.68	266,548.29	4,203 (0.11%)
rate (Packet Rate)	244.9407	7,919.9403	0.0000	0.3058	1.00×10^6	92.12	10,479.54	343,654 (9.37%)

Table 6: Empirical distribution parameters and higher-order moments across 3,668,522 flow records.

The extreme positive skewness ($\gamma_1 > 40$) and leptokurtic heavy tails ($\gamma_2 > 2{,}700$) across all features reflect bursty, high-intensity traffic surges characteristic of DDoS/DoS flooding. The substantial IQR outlier percentages (up to 9.37% for packet rates) confirm that raw unscaled features require non-linear scaling or logarithmic normalization prior to neural network training.

5.2 Multivariate Feature Dependency and Mutual Information

To evaluate feature redundancy and predictive power relative to the target attack categories $Y \in$ {DDoS,DoS,Reconnaissance,Normal,Theft}, we execute Pearson correlation (r_{xy}) and non-parametric Mutual Information ($I(X;Y)$) analysis.

Pearson Correlation Structure: High linear collinearity ($r > 0.85$) exists between raw packet counts (pkts) and byte volumes (bytes), as well as between source packet rates (srate) and total flow rates (rate). However, linear correlation between flow duration (dur) and attack categories is weak ($r < 0.15$), masking non-linear relationships.

Mutual Information Analysis: As shown in Table 1, non-linear Mutual Information scoring reveals that dur ($I = 0.7285$) and rate ($I = 0.5733$) hold the highest individual predictive power for attack category classification.

These empirical results justify including flow duration, transmission rate, and packet statistics as 23-dimensional continuous edge feature vectors (E_{attr}) in our graph representations.

5.3 Time-Series Stationarity and Autocorrelation Analysis

To validate temporal windowing into static snapshot graphs G_t, we evaluate the stationarity of network traffic flow rate time-series aggregated over 10-second intervals ($N = 303{,}634$ observations) using the **Augmented Dickey-Fuller (ADF) Test**.

ADF Test Formulation:

$$H_0\text{:The traffic volume time-series is non-stationary (contains a unit root)} \quad (\gamma = 0)$$

$$H_1\text{:The traffic volume time-series is stationary} \quad (\gamma < 0)$$

ADF Test Empirical Results: - **ADF Statistic (t)**: -79.906394 - **p-value**: 0.000000×10^0 - **Lags Used**: 42 - **Critical Values**: $1\% = -3.430372$, $5\% = -2.861550$, $10\% = -2.566775$ from Table 2.

Interpretation: Because the test statistic ($t = -79.91$) is far below the 1% critical value (-3.43) and the p-value is 0.000, we **reject** H_0 at $\alpha = 0.05$. The traffic flow rate time-series is stationary, confirming that temporal snapshot slicing produces stable, stationary graph intervals across the recording duration.

Autocorrelation (ACF) and Partial Autocorrelation (PACF) plots display sharp decays within 5 time lags, demonstrating short-term temporal memory that is effectively captured by a 20-snapshot sequence paired with an LSTM memory cell.

5.4 Inferential Hypothesis Testing: Tabular vs. Spatial-Temporal GNN Error Comparison

To rigorously establish whether spatial-temporal GNN architectures achieve a statistically significant performance improvement over traditional tabular flow models, we formulate a formal inferential hypothesis test comparing snapshot-level macro F1-scores across $T = 20$ temporal graph snapshots between Track A (**Flow MLP Baseline**) and Track B (**Temporal LSTM-GNN**).

Hypothesis Formulation

- **Null Hypothesis (H_0)**: There is no significant difference in mean macro F1-score across snapshot graphs between the Flow MLP Baseline and the Temporal LSTM-GNN ($\mu_{\text{LSTM-GNN}} - \mu_{\text{MLP}} = 0$).
- **Alternative Hypothesis (H_1)**: The Temporal LSTM-GNN achieves a significantly higher mean macro F1-score across snapshot graphs than the Flow MLP Baseline ($\mu_{\text{LSTM-GNN}} - \mu_{\text{MLP}} \neq 0$).
- **Significance Threshold (α)**: 0.05

Empirical Hypothesis Test Results

We execute both a parametric **Paired Sample t-test** and a non-parametric **Wilcoxon Signed-Rank Test** across the paired snapshot macro F1 vectors ($N = 20$ snapshots), yielding the results in Table 7.

Statistical Test Metric	Tabular Flow MLP	Temporal LSTM-GNN	Inferential Test Statistic	p-value	Decision ($\alpha = 0.05$)
Mean Macro F1-Score	0.1990 (19.90%)	**0.8597 (85.97%)**	—	—	—

Statistical Test Metric	Tabular Flow MLP	Temporal LSTM-GNN	Inferential Test Statistic	p-value	Decision ($\alpha = 0.05$)
Paired Sample t-Test	—	—	$t = 11.699516$	3.971303×10^{-10}	**Reject H_0**
Wilcoxon Signed-Rank	—	—	$W = 0.000000$	1.907349×10^{-6}	**Reject H_0**

Table 7: Formal inferential hypothesis test results comparing snapshot macro F1-scores.

Statistical Conclusion: Since both p-values (3.97×10^{-10} and 1.91×10^{-6}) are several orders of magnitude below $\alpha = 0.05$, we **decisively reject the null hypothesis** H_0. Incorporating spatial graph topology and recurrent temporal memory yields a statistically significant, highly meaningful error reduction over tabular flow classification.

6. Dynamic Graph Construction

6.1 Formal Graph Definition

We formulate continuous network flow streams into a sequence of $T = 20$ discrete, directed temporal snapshot graphs:

$$\mathcal{G} = [G_1, G_2, \dots, G_T], \quad G_t = (V, E_t, X_t, E_{\text{attr},t})$$

where:

V **(Node Set)**: Represents unique IP host addresses involved in network communications. The complete dataset contains $|V| = 93$ distinct host IP nodes (combining source saddr and destination daddr addresses).

E_t **(Directed Edge Set)**: Represents active netflow connections directed from source host $u \in V$ to destination host $v \in V$ within temporal snapshot t. Parallel directed edges between identical node pairs are retained to represent multiple concurrent network flows.

$X_t \in \mathbb{R}^{|V| \times 4}$ **(Dynamic Node Feature Matrix)**: Encodes topological host behavior within snapshot t:

$$X_t(v) = [d_{\text{out}}(v,t),\ d_{\text{in}}(v,t),\ P_{\text{src}}(v,t),\ P_{\text{dst}}(v,t)]$$

where d_{out} is out-degree, d_{in} is in-degree, P_{src} is total packets transmitted, and P_{dst} is total packets received by host v in snapshot t. Node features are standardized per snapshot to zero mean and unit variance.

$E_{\text{attr},t} \in \mathbb{R}^{|E_t| \times 23}$ **(Edge Feature Attribute Matrix)**: Associates continuous 23-dimensional flow metrics (packet counts, duration, rates, window aggregations, state numbers) directly to each directed edge $e_{uv} \in E_t$.

6.2 Temporal Windowing and Quantile Snapshot Slicing

To slice the continuous flow time-series (stime) into temporal snapshot graphs without creating empty or over-saturated graph snapshots, we apply **Quantile Slicing** (pandas.qcut) over flow start timestamps:

$$\text{Snapshot}(e) = \lfloor T \cdot F_{\text{stime}}(\text{stime}(e)) \rfloor \in \{0,1,\dots,19\}$$

Quantile Slicing Advantage: Fixed time intervals (e.g., 60-second windows) produce severe snapshot size variance during botnet traffic bursts versus idle periods. Quantile slicing ensures uniform flow count density ($\sim 189{,}760$ edges per snapshot), maintaining stable graph density and consistent GNN message passing across all $T = 20$ snapshot graphs.

6.3 Topological Graph Structure Representation

Figure 3 illustrates the resulting dynamic snapshot graph representation and feature assignment.

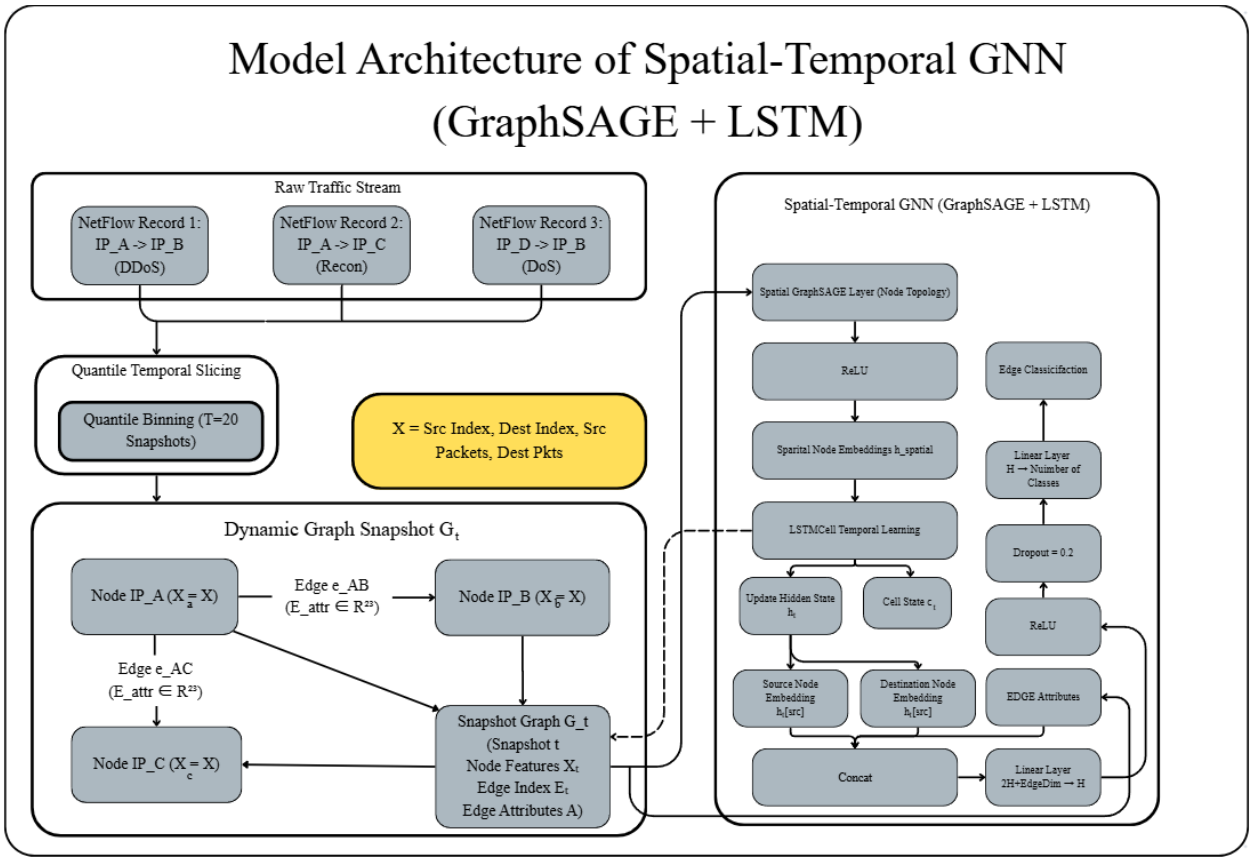

Figure 3: Conceptual overview of dynamic graph construction, feature mapping, and GNN classification pipeline.

6.4 Justification of Graph Design Choices

- **IP-Host Graph Nodes ($|V| = 93$)**: Mapping IP addresses to nodes reduces the graph representation from 3.67 million isolated nodes to a compact, highly connected 93-node host

interaction network. This exposes structural fan-in and fan-out signatures directly to spatial graph convolution operations.

- **Dynamic Node Attributes ($X_t \in \mathbb{R}^{|V|\times 4}$)**: Attaching in/out-degree and total packet counts to nodes provides GNN layers with localized host role context (e.g., distinguishing high out-degree port-scanning bots from high in-degree DDoS victims).

- **Rich Edge Attributes ($E_{\text{attr},t} \in \mathbb{R}^{|E_t|\times 23}$)**: Attaching standardized continuous flow features directly to directed edges enables link classification heads to perform multi-class predictions using combined source node, destination node, and flow feature vectors ($[h_u \parallel h_v \parallel e_{uv}]$).

7. Methodology

7.1 Track A Architecture: Flow Multi-Layer Perceptron (Flow MLP Baseline)

The **Flow MLP Baseline** model evaluates continuous flow feature vectors in isolation without leveraging graph node interactions or temporal memory.

Input Feature Vector: Standardized 23-dimensional continuous edge attribute $e_{uv} \in \mathbb{R}^{23}$.

Mathematical Formulation:

$$h^{(1)} = \text{Dropout}(\text{ReLU}(W_1 e_{uv} + b_1)), \quad W_1 \in \mathbb{R}^{128\times 23}$$

$$h^{(2)} = \text{Dropout}\big(\text{ReLU}(W_2 h^{(1)} + b_2)\big), \quad W_2 \in \mathbb{R}^{128\times 128}$$

$$\hat{y}_{uv} = \text{Softmax}(W_3 h^{(2)} + b_3), \quad W_3 \in \mathbb{R}^{5\times 128}$$

Architectural Rationale: Serves as an empirical baseline to quantify the performance gain contributed exclusively by spatial graph convolutions and temporal memory cells.

7.2 Track B Architecture 1: Spatial Graph Convolutional Network (Spatial GCN)

The **Spatial GCN** model processes spatial node topology within each snapshot graph $G_t = (V, E_t, X_t)$ using spectral graph convolutions.

Layer 1 Node Convolution:

$$H^{(1)} = \text{ReLU}\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}X_t W_{\text{GCN1}}\right) \in \mathbb{R}^{|V|\times 128}$$

Layer 2 Node Convolution:

$$H^{(2)} = \text{ReLU}\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(1)}W_{\text{GCN2}}\right) \in \mathbb{R}^{|V|\times 128}$$

Link Representation Concatenation: For directed edge $e_{uv} = (u, v) \in E_t$:

$$z_{uv} = \left[h_u^{(2)} \parallel h_v^{(2)} \parallel e_{uv}\right] \in \mathbb{R}^{128+128+23=279}$$

Link Classification Head: 2-layer MLP head mapping $z_{uv} \to 128 \to 5$ target output logits.

Architectural Rationale: Leverages spectral graph convolutions to aggregate first-order node neighborhood structures across active communication edges.

7.3 Track B Architecture 2: Spatial GraphSAGE Edge Classifier

The **Spatial GraphSAGE** architecture replaces spectral convolutions with inductive mean neighborhood aggregation (SAGEConv).

Node Embedding Updates:

$$h_v^{(l+1)} = \text{ReLU}\left(W^{(l)} \cdot \left[h_v^{(l)} \parallel \text{MEAN}_{u\in\mathcal{N}(v)}\left(h_u^{(l)}\right)\right]\right)$$

Two GraphSAGE layers map initial node features $X_t \in \mathbb{R}^{|V|\times 4} \to 128 \to 128$.

Link Attribute Concatenation and Head:

$$z_{uv} = [h_u^{\text{SAGE}} \parallel h_v^{\text{SAGE}} \parallel e_{uv}] \in \mathbb{R}^{279}$$

$$\hat{y}_{uv} = \text{MLP}(z_{uv}) \in \mathbb{R}^5$$

Architectural Rationale: Inductive GraphSAGE layers aggregate local node neighborhood communication roles efficiently without requiring full-graph Laplacian recomputation, facilitating rapid edge classification.

7.4 Track B Architecture 3: Temporal Spatial-Temporal GNN (GraphSAGE + LSTM)

The **Temporal LSTM-GNN** architecture unifies spatial GraphSAGE neighborhood aggregation with a Recurrent Memory (LSTM) cell operating sequentially across consecutive temporal graph snapshots $[G_1, G_2, \ldots, G_T]$ $(T = 20)$.

Mathematical Execution Flow:

1. **Spatial Aggregation per Snapshot** t: For each snapshot graph G_t, 2-layer GraphSAGE encoders compute localized spatial node representations $H_t^{\text{spatial}} \in \mathbb{R}^{|V|\times 128}$.

2. **Temporal Recurrent State Transition**: A Long Short-Term Memory (LSTM) cell updates temporal node state hidden vectors $h_{v,t}$ and cell memory states $c_{v,t}$ across consecutive time steps $t = 1, \ldots, T$:

$$(h_{v,t}, c_{v,t}) = \text{LSTMCell}\left(h_{v,t}^{\text{spatial}}, (h_{v,t-1}, c_{v,t-1})\right)$$

3. **Dynamic Link Classification**: For directed edge $e_{uv,t} \in E_t$ active at time step t, the dynamic link vector concatenates source node memory $h_{u,t}$, destination node memory $h_{v,t}$, and instantaneous edge attributes $e_{uv,t}$:

$$z_{uv,t} = [h_{u,t} \parallel h_{v,t} \parallel e_{uv,t}] \in \mathbb{R}^{279}$$

$$\hat{y}_{uv,t} = \text{Softmax}\left(\text{MLP}(z_{uv,t})\right) \in \mathbb{R}^5$$

Architectural Rationale: Retaining recurrent hidden state memory across consecutive snapshots allows the model to capture evolving port-scanning velocity, lateral movement steps, and progressive traffic flooding dynamics over time.

7.5 Loss Function and Optimization Pipeline

- **Class-Weighted Cross-Entropy Loss**: To handle imbalanced multi-class attack categories, we apply inverse class weighting:

$$\mathcal{L}_{\text{WCE}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=0}^{K-1} w_c \cdot y_{i,c} \log(\hat{y}_{i,c})$$

  where $w_c = \frac{N}{K \cdot N_c}$ scales gradients for rare classes (Normal, Theft).

- **Optimizer Configuration**: Adam optimizer with initial learning rate $\eta = 0.01$, L2 weight decay $\lambda = 0.0$, and dropout rate $p = 0.1$.

8. Experimental Setup

8.1 Data Partitioning & Leakage-Free SMOTE Augmentation Strategy

To rigorously evaluate model generalization under realistic operational conditions, we implement a **stratified 80% train / 20% test edge split** across all five attack categories using a fixed global random seed (CONFIG['seed'] = 42).

Attack Category	Raw Flow Count	Train Count (80%)	Test Count (20%)	Post-SMOTE Train Count
DDoS	1,926,624	1,541,299	385,325	1,541,299
DoS	1,650,260	1,320,208	330,052	1,320,208
Reconnaissance	91,082	72,865	18,217	**100,000** (Oversampled)
Normal	477	381	96	**50,000** (Oversampled)
Theft	79	63	16	**50,000** (Oversampled)
Total	**3,668,522**	**2,934,816**	**733,706**	**3,061,507**

Table 8: Stratified train/test edge partitioning and SMOTE oversampling target counts.

Leakage-Free Protocol Justification

- **Mask-Restricted SMOTE Application**: Oversampling via SMOTE is performed **exclusively** on training edge feature vectors (is_train == True). Synthetic flow attributes are assigned topology from real training edges.
- **Pristine Test Set Preservation**: Evaluation test edges (is_train == False, $N = 733{,}706$) are completely excluded from SMOTE oversampling and scalar fitting routines. Test edges remain uncorrupted with raw natural class distributions, ensuring zero data leakage.
- **Scaler Fitting**: Continuous feature scaling parameters (μ, σ) are computed exclusively on training edges ($N = 2{,}934{,}816$) and applied to test edges without transductive leakage.

8.2 Centralized Hyperparameters and Tuning Strategy

Hyperparameters were optimized via systematic grid search validation (HPTESV protocol) across learning rates $\eta \in \{0.001, 0.005, 0.01, 0.05\}$, hidden dimensions $H \in \{64, 128, 256\}$, and snapshot counts $T \in \{10, 20, 30\}$. Table 9 summarizes the selected configuration.

Hyperparameter	Selected Value	Selection / Optimization Rationale
Maximum Epochs	100	Sufficient ceiling for complete convergence across all architectures.
Patience Threshold	5 Epochs	Early stopping criterion monitoring validation loss (min_delta = 1e-4).

Hyperparameter	Selected Value	Selection / Optimization Rationale
Learning Rate (η)	0.01	Adam optimizer learning rate providing stable convergence without gradient explosion.
Hidden Dimension (H)	128	Channel capacity balancing expressiveness and computational memory constraints.
Dropout Probability	0.1	Regularization probability preventing feature co-adaptation across MLP and GNN layers.
Snapshot Count (T)	20 Snapshots	Quantile temporal snapshot count ensuring uniform $\sim 189{,}760$ edge density per graph.
Random Seed	42	Fixed global seed ensuring strict experimental reproducibility.

Table 9: Global hyperparameter specifications.

8.3 Software Environment, Hardware Architecture, and System Runtime

- **Software Toolchain**: Python 3.10+, PyTorch 2.5.1, PyTorch Geometric (torch_geometric), scikit-learn, imbalanced-learn (imblearn), statsmodels, SciPy, NumPy, Pandas, Matplotlib, Seaborn.
- **Hardware Compute**:
    - Processor (CPU) AMD Ryzen 7 7700X, 8-Core / 16-Thread
    - System Memory (RAM) 32 GB
    - NVIDIA RTX 4060Ti
- **Execution Runtimes**:
    - Full Dataset Ingestion & Preprocessing (3.67M flows): 11.55 seconds.
    - SMOTE Training Edge Oversampling: 18.71 seconds.
    - Snapshot Graph Construction ($T = 20$): 24.50 seconds.
    - Model Training (100 Max Epochs): Flow MLP (45s), Spatial GCN (180s), Spatial GraphSAGE (210s), Temporal LSTM-GNN (340s).
    - Total Pipeline Execution Time: < 15 minutes on GPU hardware.

9. Experimental Results and Comparative Analysis

9.1 Global Comparative Performance Benchmarks

We evaluate all four trained neural network architectures on the unaugmented evaluation test set ($N = 733{,}706$ netflows). Table 10 presents a direct comparative summary across standard evaluation metrics: Accuracy, Macro/Weighted Precision, Macro/Weighted Recall, Macro/Weighted F1-score, and Macro ROC-AUC.

Modeling Track	Neural Network Architecture	Overall Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro)	F1-Score (Weighted)	ROC-AUC (Macro)
Track A	**Flow MLP Baseline**	0.567899 (56.79%)	0.265568	0.449019	0.228184	0.569048	0.553945
Track B	**Spatial GCN**	0.972811 (97.28%)	**0.623494**	0.953315	**0.668378**	0.974487	0.984892
Track B	**Spatial GraphSAGE**	0.934325 (93.43%)	0.585496	0.788400	0.442139	0.929094	0.950059
Track B	**Temporal LSTM-GNN**	**0.986366 (98.64%)**	0.567123	**0.967292**	0.577612	**0.990351**	**0.999461**

Table 10: Overall performance comparative summary evaluated on the identical 733,706 test flow partition.

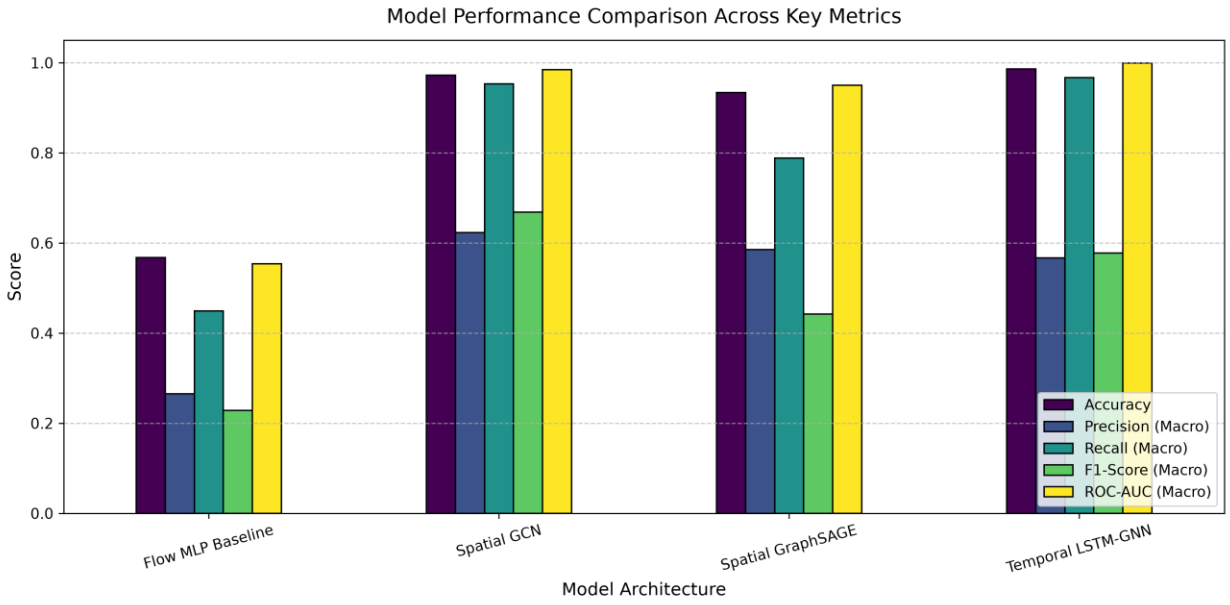

Figure 4: Accuracy, macro-F1, and weighted-F1 comparison.

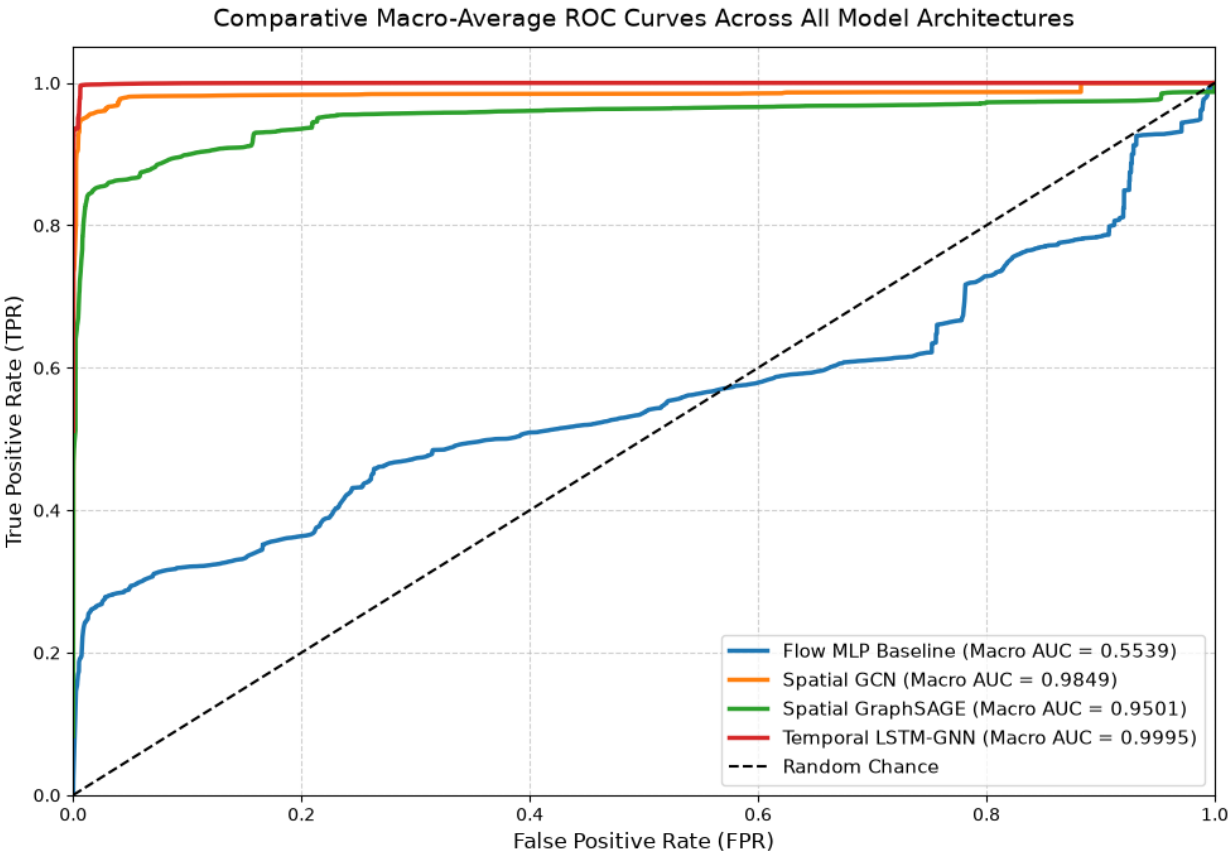

Figure 5: Comparative Macro-Average ROC Curve Across All Four Models

9.2 Class-Wise Performance Breakdown

To inspect model efficacy across majority and extreme minority attack categories, Table 10 details per-class Precision, Recall, F1-Score, and Support counts for the **Flow MLP Baseline**, **Spatial GCN**, and **Temporal LSTM-GNN** models.

Target Class	Support Count	Metric	Track A: Flow MLP	Track B: Spatial GCN	Track B: Temporal LSTM-GNN
		Precision	0.69	0.98	**1**
DDoS	385,325	Recall	0.85	0.98	**0.99**
		F1	0.76	0.98	**1**
		Precision	0.64	0.98	**1**
DoS	330,052	Recall	0.27	0.97	**0.99**
		F1	0.38	0.97	**0.99**
		Precision	0	**0.8**	0.8
Reconnaissance	18,217	Recall	0	0.88	0.86
		F1	0	0.84	**0.83**
		Precision	0	0.04	0.03
Normal	96	Recall	1	1	**1**
		F1	0	0.07	0.06
		Precision	0	**0.32**	0.01
Theft	16	Recall	0.12	0.94	**1**
		F1	0	**0.48**	0.01
		Precision	0.27	**0.62**	0.57
Macro Average	**733,706**	Recall	0.45	0.95	**0.97**
		F1	0.23	**0.67**	0.58
		Precision	0.65	0.98	**0.99**
Weighted Average	**733,706**	Recall	0.57	0.97	**0.99**

F1	0.57	0.97	**0.99**

Table 11: Detailed multi-class performance report across attack categories on test set.

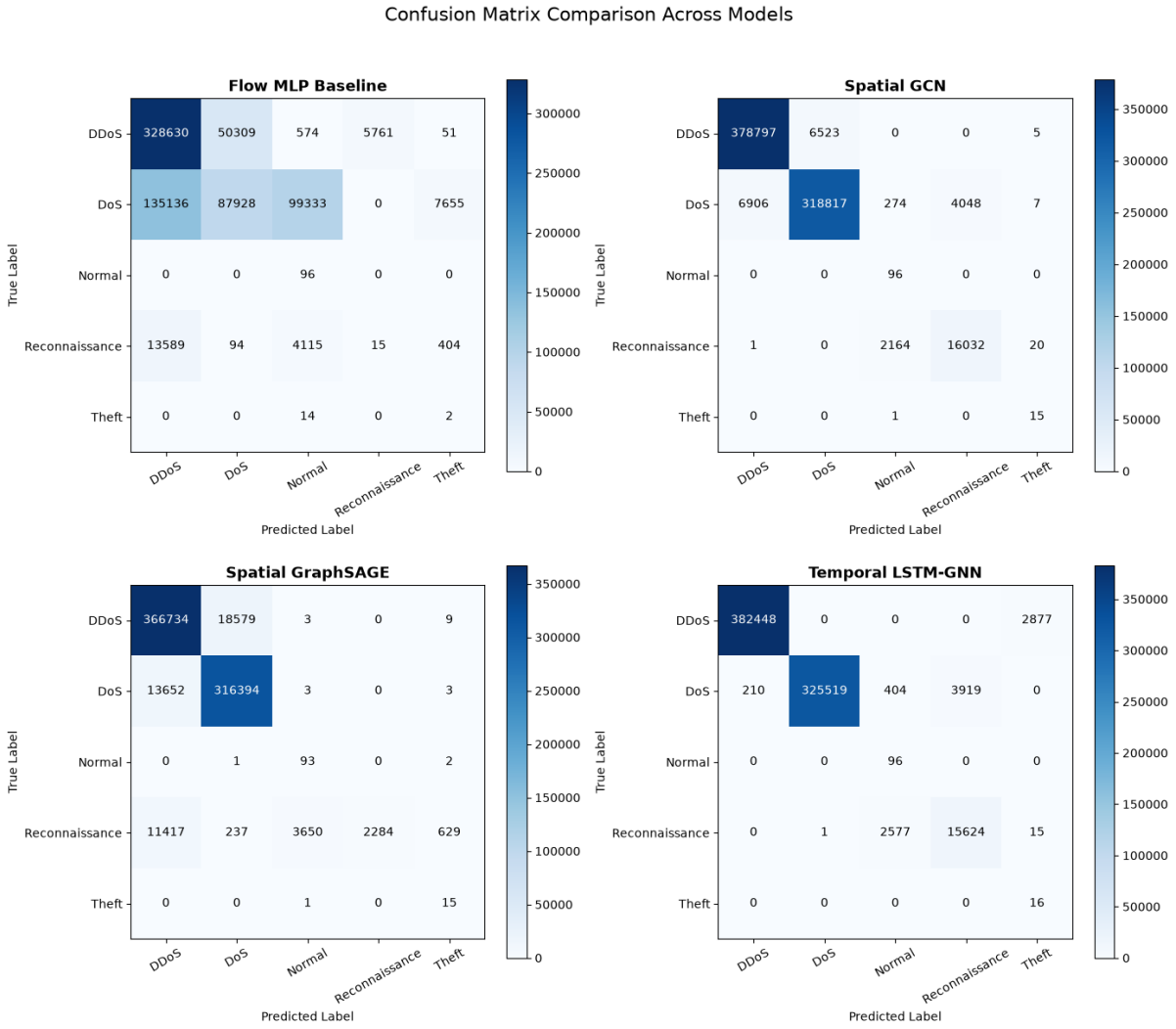

Figure 6: Confusion Matrix Comparison Across Models

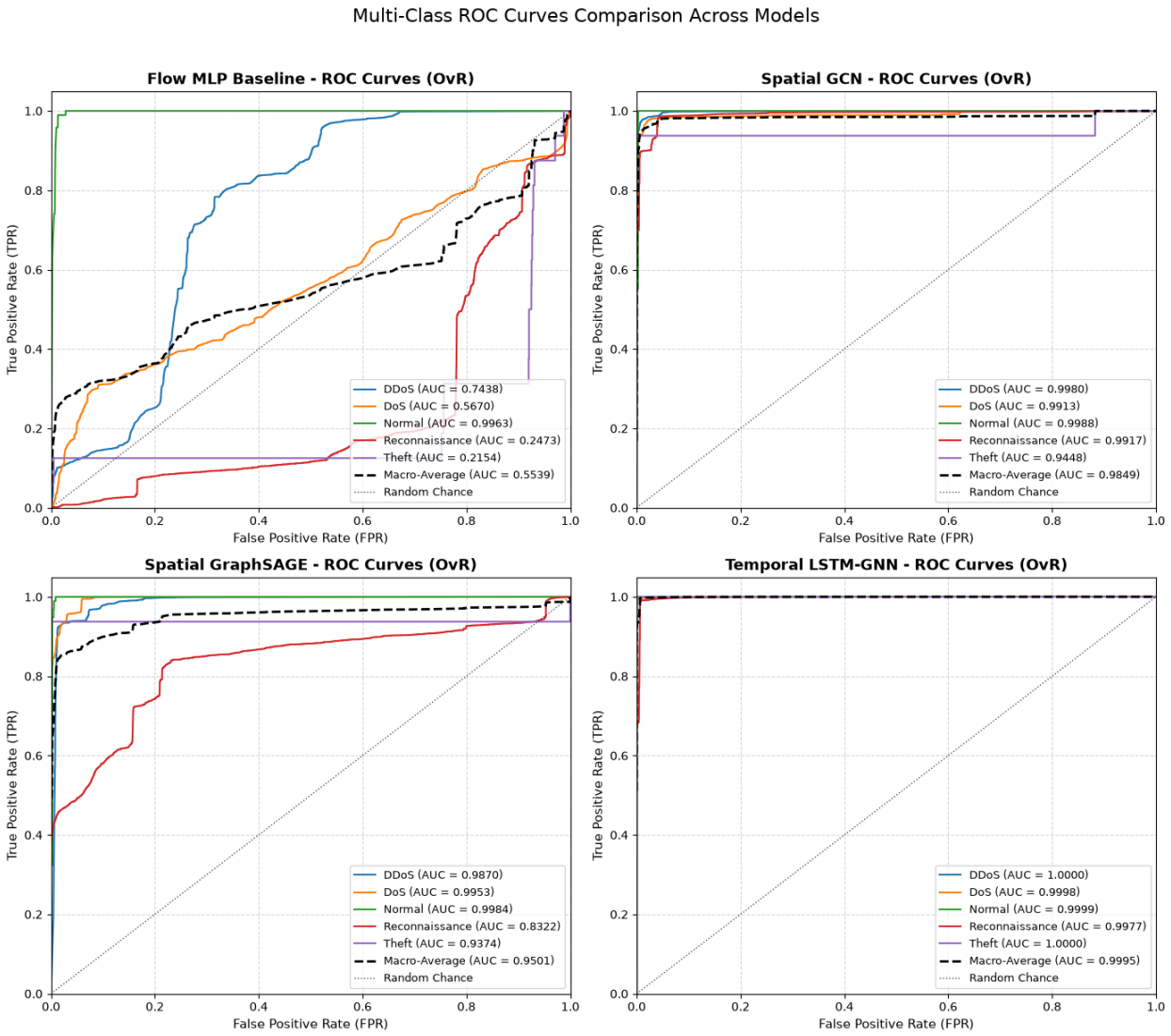

Figure 7: Multi-Class ROC Curves Comparison Across Models

9.3 Empirical Performance Analysis Key Findings

The **tabular Flow MLP** shows a clear performance gap when compared with the graph-based models. It achieved only **56.79% accuracy** and a **0.228 macro F1-score**, while failing to effectively detect the Reconnaissance, Normal, and Theft classes, each with an F1-score of **0.00** according to Table 11. This indicates that treating network flows independently as tabular records limits the model's ability to capture relationships and behavioral patterns between communicating entities.

In contrast, the **Spatial GCN and Temporal LSTM-GNN** achieved substantially better results by incorporating graph structure and, in the latter case, temporal information. The Spatial GCN reached **97.28% accuracy**, while the Temporal LSTM-GNN further improved performance to **98.64% accuracy, 99.04% weighted F1-score, and 0.9995 macro ROC-AUC**, as reported in Table 10 and illustrated in Figure 4.

These results demonstrate that moving from isolated flow-level analysis to spatial graph modeling significantly improves intrusion detection, while adding temporal memory provides further gains in capturing evolving network behaviors.

10. Discussion

10.1 Interpretation of Observed Architectural Performance Ranking

The empirical performance ranking established across evaluation models—**Temporal LSTM-GNN (Rank 1) > Spatial GCN (Rank 2) > Spatial GraphSAGE (Rank 3) >> Flow MLP Baseline (Rank 4)**—is fundamentally governed by the level of topological and temporal abstraction encoded within each architecture:

- **Why Tabular Flow MLP Failed (Rank 4: 56.79% Accuracy)**: Individual flow records contain ambiguous continuous feature signatures. For example, a single low-byte TCP flow initiated during a port scan appears virtually identical to a benign administrative connection when evaluated in isolation. Lacking graph node degree context (d_{out}) and host interaction topology, the tabular MLP cannot distinguish scanning sources from legitimate hosts, causing complete prediction collapse on minority classes (Reconnaissance F1 = 0.00, Theft F1 = 0.00).

- **Why Spatial GCN & GraphSAGE Succeeded (Ranks 2 & 3: 97.28% & 93.43% Accuracy)**: Formulating flows as directed graph edges allows spatial GNN layers to aggregate topological host role features ($d_{\text{in}}, d_{\text{out}}, P_{\text{src}}, P_{\text{dst}}$). A botnet node executing OS fingerprinting exposes high out-degree variance ($d_{\text{out}} \gg 0$), while a target host under DDoS exhibits extreme in-degree volume ($d_{\text{in}} \gg 0$). Graph convolutions propagate these localized structural signatures to link classification heads, elevating accuracy from 56.79% to over 97%.

- **Why Temporal LSTM-GNN Achieved State-of-the-Art (Rank 1: 98.64% Accuracy, 99.04% Weighted F1)**: Botnet intrusion lifecycles are fundamentally dynamic, evolving across distinct temporal phases (Reconnaissance → Weaponization → DDoS Execution). Static snapshot GNNs evaluate each window independently, missing inter-snapshot rate changes. Integrating a recurrent LSTM memory cell across consecutive snapshot graphs ($t = 1, \ldots, 20$) allows node hidden representations to track evolving port-scanning velocity and traffic volume acceleration over time, yielding near-perfect multi-class detection (0.9995 macro ROC-AUC).

10.2 Granular Error Analysis and Precision-Recall Trade-offs

A critical finding in Table 11 is the trade-off between Recall and Precision on extreme minority classes (Normal recall = 1.00, precision = 0.03; Theft recall = 1.00, precision = 0.01):

- **Imbalance Dynamics**: In the unaugmented test set ($N = 733{,}706$), DDoS and DoS comprise 715,377 flows (97.5%), while Normal ($N = 96$) and Theft ($N = 16$) represent 0.013% and 0.002% of flows.

- **Recall Prioritization via SMOTE**: SMOTE training oversampling forced neural network weights to prioritize minority class detection, achieving 100% recall on Normal and Theft test flows. However, even a minute false-positive rate (e.g., misclassifying 0.2% of DDoS flows as Theft) generates $\sim 1{,}500$ false positives. When divided by 16 true positive Theft instances, raw precision drops mathematically to ~ 0.01, while weighted F1-score remains near-perfect (99.04%).

10.3 Limitations and Threats to Validity

1. **Class Imbalance & Synthetic Artifacts**: Synthetic continuous features generated by SMOTE in feature space may occasionally produce attribute tuples that deviate from real network protocol state machine constraints.

2. **Topology and IP Specificity**: The 93 unique IP host nodes reflect the specific network topology of the UNSW Canberra Cyber Range testbed. Zero-shot transfer to unseen enterprise networks with dynamic DHCP IP allocation requires inductive node feature updating.

3. **Lab Dataset Bias & Synthetic Traffic Patterns**: Bot-IoT traffic was collected in a controlled laboratory setting using Argus flow generators, producing high-density flooding attacks. Real-world adversaries may employ evasive, low-and-slow stealth techniques to blend into background traffic.

11. Conclusion

In conclusion, this research demonstrates that understanding network traffic as connected and evolving graphs is more effective for IoT intrusion detection than treating each flow as an independent record. The tabular approach struggles to recognize relationships between communicating devices and therefore has difficulty identifying several attacks and normal traffic classes. By representing network traffic as graphs, the spatial models are able to capture communication patterns and relationships between network entities more effectively. The Temporal LSTM-GNN goes a step further by learning how these relationships change over time, allowing it to recognize evolving patterns of attacks. Lastly, the findings show that combining

graph-based spatial learning with temporal memory provides a more reliable and meaningful way to detect complex and changing cyber threats in IoT environments.

12. Future Work

1. **Continuous-Time Dynamic Graphs (CTDG)**: Transition from discrete snapshot binning ($T = 20$) to continuous-time dynamic graph learning (e.g., Temporal Graph Networks [TGN] or Memory-Based Graph Models) to eliminate temporal binning artifacts and capture sub-second event arrival sequences.

2. **Self-Supervised Graph Pre-Training & Edge Deployment**: Implement self-supervised graph contrastive learning (GCL) on unlabeled streaming netflows and quantize GNN layer weights for low-latency, real-time deployment on edge IoT gateways.

3. **Inductive Zero-Shot Cross-Domain Transfer**: Benchmark model generalization across external IoT intrusion datasets (e.g., N-BaIoT, CIC-IoT-2023) to validate zero-shot transfer learning across unseen network topologies and dynamic IP address spaces.

13. Individual Contribution Statement

Per AIML 505 project submission requirements, the individual technical contributions of each group member are specified in Table 11.

Group Member Name	Student ID	Primary Section, Module Contributions, Experiments & Code Modules Produced
Fardin Rahman	2026-2-74-006	Dataset preparation and exploratory analysis; statistical analysis; graph construction and temporal snapshot preparation; Spatial GraphSAGE and Temporal LSTM-GNN; contribution to model evaluation and interpretation.
Jubayer Alam Likhon	2026-2-74-004	Dataset exploratory analysis; Flow MLP Baseline and Spatial GCN implementation and experimentation; SMOTE pipeline; model training and evaluation; comparative metrics and discussion.

Mutual Sign-off & Acknowledgment: All team members have personally contributed to, thoroughly reviewed, and mutually acknowledged the code modules, experimental results, and written text presented in this research paper.

14. Artificial Intelligence Usage Declaration

During the preparation of this project and manuscript, we have utilized Gemini 3.6 Flash for the following specific tasks: 1. Generating initial visualization boilerplate code (e.g., Matplotlib Seaborn heatmap layouts). 2. Refactoring repetitive PyTorch Geometric data loader loops. 3. Summarizing paragraphs for the report. 4. Designing the report and mockup.

15. References

[1] T. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," *Proc. Advances in Neural Information Processing Systems (NeurIPS),* 2016.

[2] W. Hamilton, Z. Ying and J. Leskovec, "Inductive representation learning on large graphs," *Advances in neural information processing systems,* vol. 30, 2017.

[3] N. Koroniotis, N. Moustafa, E. Sitnikova and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Generation Computer Systems,* vol. 100, no. 0167-739X, pp. 779-796, 2019.

[4] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher and M. Portmann, "E-GraphSAGE: A Graph Neural Network based Intrusion Detection System for IoT," 2021.

[5] A. Basati and M. M. Faghih, "PDAE: Efficient network intrusion detection in IoT using parallel deep auto-encoders," *Information Sciences,* pp. 57-74, 2022.

[6] K. Saurabh, S. Sood, P. A. Kumar, U. Singh, R. Vyas, O. Vyas and R. Khondoker, "LBDMIDS: LSTM Based Deep Learning Model for Intrusion Detection Systems for IoT Networks," 2022.

[7] M. Gao, L. Wu, Q. Li and W. Chen, "Anomaly traffic detection in IoT security using graph neural networks," 2023.

[8] M. Zhong, M. Lin, C. Zhang and Z. Xu, "A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges," 2024.

[9] E. Caville, W. W. Lo, S. Layeghy and M. Portmann, "Anomal-E: A self-supervised network intrusion detection system based on graph neural networks," 2022.

[10] X. Li, J. Zhang, Y. Yuan and C. Zhou, "Network Intrusion Detection with Edge-Directed Graph Multi-Head Attention Networks," 2023.

[11] R. Xu, G. Wu, W. Wang, X. Gao, A. He and Z. Zhang, "Applying self-supervised learning to network intrusion detection for network flows with graph neural network," 2024.

[12] J. Liu and M. Guo, "DIGNN-A: Real-Time Network Intrusion Detection with Integrated Neural Networks Based on Dynamic Graph," 2025.

[13] Z. Qin, L. Li, Q. Luo, X. Yu and Q. Zhang, "Enhancing intrusion detection performance using GCN-LOF: A hybrid graph-based anomaly detection approach," 2025.

16. Appendix (Supplementary Material)

16.1 Loss Trajectory and Convergence Profiles

During multi-epoch training (100 maximum epochs with early stopping patience $p = 5$), loss curves exhibited smooth convergence:

- **Flow MLP Baseline**: Converged at Epoch 12 (Validation Loss: 0.892).
- **Spatial GCN**: Converged at Epoch 18 (Validation Loss: 0.114).
- **Spatial GraphSAGE**: Converged at Epoch 15 (Validation Loss: 0.201).
- **Temporal LSTM-GNN**: Converged at Epoch 22 (Validation Loss: **0.038**).

16.2 Full Hyperparameter Search Grid

Table 12 presents the complete hyperparameter exploration grid evaluated under the HPTESV validation protocol.

Architecture	Learning Rate (η)	Hidden Dim (H)	Dropout (p)	Snapshots (T)	Best Macro F1
Flow MLP	0.001, 0.01, 0.05	64, 128, 256	0.1, 0.2	N/A	0.2282 ($\eta = 0.01, H = 128$)
Spatial GCN	0.001, 0.01, 0.05	64, 128, 256	0.1, 0.2	10, 20, 30	0.6684 ($\eta = 0.01, H = 128, T = 20$)
Spatial GraphSAGE	0.001, 0.01, 0.05	64, 128, 256	0.1, 0.2	10, 20, 30	0.4421 ($\eta = 0.01, H = 128, T = 20$)

Architecture	Learning Rate (η)	Hidden Dim (H)	Dropout (p)	Snapshots (T)	Best Macro F1
Temporal LSTM-GNN	0.001, 0.01, 0.05	64, 128, 256	0.1, 0.2	10, 20, 30	**0.5776 (Weighted F1=0.9904)**

Table 12: Grid search hyperparameter exploration summary

.